第 4 节 Docker 容器技术

Docker 官方的定义：Docker 是一个用于开发、发布和运行应用程序的开放平台。Docker 可将应用程序与基础架构分离，以便快速交付软件。借助 Docker，我们可以像管理应用程序一样管理基础架构。通过利用 Docker 的发布、测试和部署代码方法，可以显著减少编写代码和在生产中运行代码之间的延迟。

本节课讲授 Docker 的安装、架构和内部原理。

4.1 Docker 的安装

1．添加 Docker 仓库

```
sudo apt-get update
sudo apt-get install -y \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sudo
gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] https://mirrors.aliyun.com/docker-
ce/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

2．安装 Docker CE

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-
compose-plugin
```

3．添加 Docker 镜像加速器

```
sudo mkdir -p /etc/docker
sudo tee /etc/docker/daemon.json <<-'EOF'
```

```
{
  "registry-mirrors": ["https://tehurc7d.mirror.aliyuncs.com"]
}
EOF
sudo systemctl daemon-reload
sudo systemctl restart docker
docker info | grep aliyun
```

4．CRI-Docker 部署

部署完 Docker CE 之后，还需要 cri-docker shim 才可以和 Kubernetes 集成。

```
wget https://github.com/Mirantis/cri-dockerd/releases/download/v0.3.15/cri-dockerd_0.3.15.3-0.ubuntu-focal_amd64.deb
dpkg -i cri-dockerd_0.3.15.3-0.ubuntu-focal_amd64.deb
sed -i 's/ExecStart=.*/ExecStart=\/usr\/bin\/cri-dockerd --container-runtime-endpoint fd:\/\/ --network-plugin=cni --pod-infra-container-image=registry.cn-hangzhou.aliyuncs.com\/google_containers\/pause:3.9/' /lib/systemd/system/cri-docker.service
systemctl daemon-reload
systemctl restart cri-docker.service
systemctl enable cri-docker.service
```

5．创建第一个容器

```
docker run -d -p 8000:80 --name container1 nginx
docker ps
```

```
root@cka-worker1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                       NAMES
428647f4a903   nginx     "/docker-entrypoint.…"   15 minutes ago   Up 15 minutes   0.0.0.0:8000→80/tcp, [::]:8000→80/tcp   container1
root@cka-worker1:~#
```

进入容器，修改 nginx 主页：（注意：每个容器的 ID 都不相同。）

```
docker exec -it container1 /bin/bash
root@ 428647f4a903:/# echo hello container, 2025! > /usr/share/nginx/html/index.html
root@ 428647f4a903:/# exit
```

访问容器内容：

```
curl http://127.0.0.1:8000
```

```
root@ae08378fd76d:/# echo hello container, 2026! > /usr/share/nginx/html/index.html
root@ae08378fd76d:/# exit
exit
root@cka-worker1:~# curl http://127.0.0.1:8000
hello container, 2026!
root@cka-worker1:~#
```

4.2 容器技术的本质

容器 = CGroup + Namespace + Rootfs + 容器引擎。

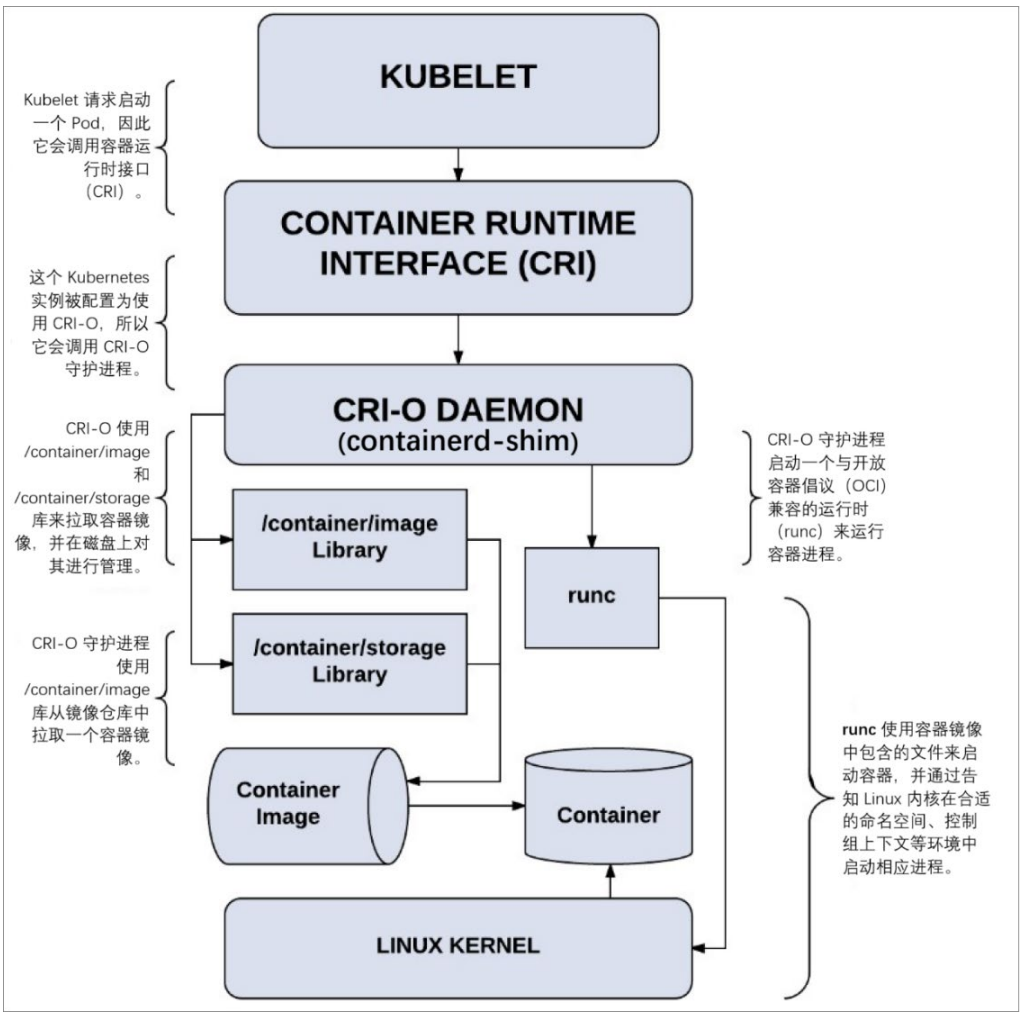
容器就是利用 Linux 内核提供的 CGroup 和 Namespace 等技术，将一系列进程隔离在相互独立的环境中进行运行。

实际上，在现代的 Linux 发行版中，任何一组进程都是容器化的：

- /proc/PID/cgroup 文件保存了进程所属的控制组。
- /proc/PID/status 文件保存了进程的 Capability 信息。
- /proc/PID/ns 保存了进程的命名空间。

容器运行时是实现容器的生命周期管理的工具：

- 容器运行时分为低层（Low-Level）运行时和高层（High-level）运行时。
- 低层运行时调整上述资源限制、安全限制和命名空间，并启动一组受限的进程。典型代表就是 runC。
- 高层运行时（容器引擎）建立在低层运行时的基础之上，提供 Web API、镜像管理、容器生命周期管理等功能。典型代表是 Containerd（以及基于 Containerd 实现的 Docker）、Podman、CRI-O。



这个流程体现了 Kubernetes "上层编排调度+下层标准化运行时"的架构设计特点。

kubelet → CRI-O → runc → Linux 内核

kubelet → containerd → containerd-shim → runc → Linux 内核

容器镜像是一个分层存储的归档文件（通常为 Tar 格式），包含运行容器所需的所有静态文件和元数据。其核心设计遵循开放容器倡议（OCI）标准，主要包含以下两部分

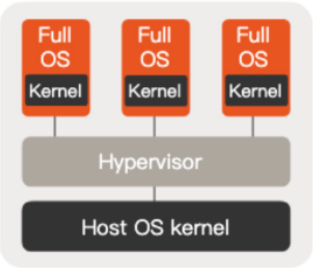
- 文件系统层（Rootfs）：提供容器的根文件系统（**rootfs**），与宿主机的文件系统隔离，是容器内进程看到的文件系统视图。
- 配置元数据（JSON 文件）：定义容器的运行时行为，包括进程配置、安全权限、环境变量等。关键文件（OCI 标准）有 **config.json**，包含容器运行时配置；**manifest.json**，描述镜像的层级结构和配置文件的路径；**layer.tar**，每个文件系统层对应的压缩包，存储实际文件内容。

按照应用场景的不同，容器分为两大类：

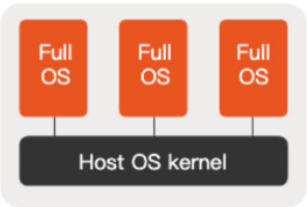
系统容器：专注于模拟系统的整个独立的用户环境，提供完整的交互体验。LXC、LXD、Incus。

应用容器：专注于提供单个服务，容器内的 Rootfs 会进行高度裁剪。Docker、Podman。

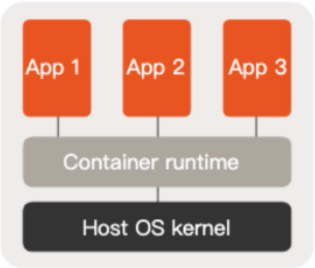
虚拟机	系统容器	应用容器
使用独立内核	使用宿主机内核	使用宿主机内核
使用了硬件虚拟化功能（Intel VT-X或AMD-V）	只使用内核功能	只使用内核功能
使用资源最多	使用资源较少	使用资源较少
有完整的引导过程，因此启动较慢	只有用户空间的init过程，启动较快	完全没有引导和init过程，启动最快
典型代表是Qemu-KVM	典型代表是LXC	典型代表是Docker



Virtual machines



System containers



Application containers

为什么要用容器？

	传统部署方式	容器化部署方式
难易程度	取决于项目	相对较为简单
部署速度	慢	敏捷
扩展性	差	好
系统扩展性	扩展性差	扩展性好
技术栈多样性	单一、封闭	多样、开放
运维	简单	运维复杂
架构复杂度	较小	复杂度高
查错	简单	定位问题困难
管理成本	开发成本	服务治理、运维

4.3 Docker 存储原理

Linux 的 Namespace、CGroups 和 UnionFS 三大技术支撑了 Docker 的实现。

Docker 使用诸多的 Linux 的隔离功能，使得容器看起来像一个轻量级虚拟机在独立运行，**容器**的本质**是**被限制了的 Namespaces 和 CGroup、并具有逻辑上独立的文件系统和网络的一个**进程**。

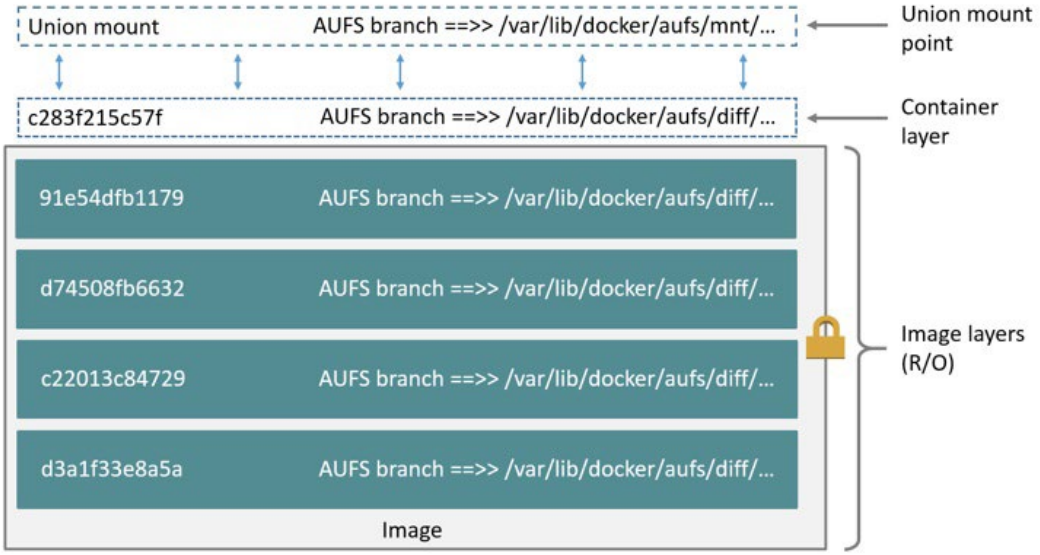
Namespace（命名空间）是容器隔离的基础，它建立了容器间的安全边界。容器间的隔离使用到的命名空间包括：User，Mnt，Network，UTS，IPC，Pid。

CGroups（Control Group 控制组）是内核功能，用于限制、记录和隔离进程组的 CPU、内存、磁盘 I/O 等资源的使用，主要的 CGroups 子系统有 cpu，blkio，device，freezer，memory。

UnionFS（联合文件系统），允许将多个不同的文件系统或目录（称为分支）合并成一个单一的、虚拟的文件系统视图，UnionFS 在资源利用、系统维护、数据管理等方面有着重要的作用。

4.3.1 UnionFS 文件系统

我们知道，Image、Container 和 Repository 是 Docker 的三个基本概念，其中 Image 就是一个包含了运行应用程序所需的所有文件、依赖和配置的 root 文件系统。Docker 的镜像通过分层机制构建，每个镜像层都是一个只读的模板，记录了所需的文件系统和配置。当执行 Docker 命令时，会基于现有镜像层创建一个新的可写层，用于构建容器运行时。这种分层机制使得镜像构建和部署变得非常高效。每条 Dockerfile 命令都会生成一个新的镜像层，使得镜像大小得以控制。Docker 使用 UnionFS 技术实现了镜像的分层存储和容器的数据持久化。UnionFS 可以将多个目录合并为一个虚拟文件系统，使得不同层级的文件和目录得以保留和共享。在容器运行时，可写层会覆盖镜像层中的文件和目录，但不会影响底层的数据，这样既保证了容器的数据持久化，又使得镜像构建变得非常高效。镜像的结构如下如所示：



以下我们使用 AUFS（Advanced UnionFS）来说明 UnionFS 的工作原理。AUFS 是 UnionFS 的升级

版，它能够提供更优秀的性能和效率

创建以下目录结构：

```
root@cka-worker1:~# mkdir aufs-test
root@cka-worker1:~# cd aufs-test/
root@cka-worker1:~/aufs-test# mkdir layer1 layer2 upperlayer mountedfs
root@cka-worker1:~/aufs-test# echo "content for file1 in layer1" > layer1/file1
root@cka-worker1:~/aufs-test# echo "content for file1 in layer2" > layer2/file1
root@cka-worker1:~/aufs-test# echo "content for file2 in layer1" > layer1/file2
root@cka-worker1:~/aufs-test# echo "content for file3 in layer2" > layer2/file3
root@cka-worker1:~/aufs-test# tree .
.
├── layer1
│   ├── file1
│   └── file2
├── layer2
│   ├── file1
│   └── file3
├── mountedfs
└── upperlayer

4 directories, 4 files
root@cka-worker1:~/aufs-test#
```

注意，layer1 和 layer2 目录中各有一个文件 file1，但文件内容不同。

使用基本语法挂载文件系统：

```
root@cka-worker1:~/aufs-test#  mount -t aufs \
> -o br:upperlayer:layer1:layer2 none mountedfs
root@cka-worker1:~/aufs-test#
```

说明：-t aufs：指定文件系统类型为 aufs。-o br:upperlayer:layer1:layer2：-o 用于指定挂载选项。br 是 aufs 的选项，表示指定分支（branches）。upperlayer:layer1:layer2 是分支的具体路径，upperlayer 是上层目录，通常是可写层。layer1 和 layer2 是只读层，通常是基础镜像层或依赖层。多个分支用冒号:分隔。none：表示没有实际的设备文件，因为 aufs 是一个虚拟文件系统。mountedfs：挂载点目录，即文件系统挂载的目标路径。

检查挂载点 mountedfs，可以看到其内容就是 layer1 和 layer2 的并集，注意 file1 的内容：

```
root@cka-worker1:~/aufs-test# cd mountedfs/
root@cka-worker1:~/aufs-test/mountedfs# ls -l
total 12
-rw-r--r-- 1 root root 28 Jan 28 00:41 file1
-rw-r--r-- 1 root root 28 Jan 28 00:41 file2
-rw-r--r-- 1 root root 28 Jan 28 00:41 file3
root@cka-worker1:~/aufs-test/mountedfs# cat *
content for file1 in layer1
content for file2 in layer1
content for file3 in layer2
root@cka-worker1:~/aufs-test/mountedfs#
```

可以看到，挂载点目录的 file1 显示的是 layer1 中 file1 的内容，而没有显示 layer2 中 file1 的内容。这是由在设置 AUFS 文件系统的分支（即层）的时候 upperlayer:layer1:layer2 选项所决定的，layer1 在前，AUFS 的优先级是从左到右的，这就意味着，如果一个文件存在于两个层中，则 AUFS 文件系统中显示的版本将是最左侧层的版本，理解这一点非常重要。

接下来，我们在挂载点目录创建一个内容为"new content for file4"的新文件 file4：

```
root@cka-worker1:~/aufs-test/mountedfs# echo "new content for file4" > file4
root@cka-worker1:~/aufs-test/mountedfs# ls -l
total 16
-rw-r--r-- 1 root root 28 Jan 28 00:41 file1
-rw-r--r-- 1 root root 28 Jan 28 00:41 file2
-rw-r--r-- 1 root root 28 Jan 28 00:41 file3
-rw-r--r-- 1 root root 22 Jan 28 01:23 file4
root@cka-worker1:~/aufs-test/mountedfs# ls -l ../upperlayer/
total 4
-rw-r--r-- 1 root root 22 Jan 28 01:23 file4
root@cka-worker1:~/aufs-test/mountedfs#
```

file4 实际是创建在了 upperlayer 这一可写层的。

继续，如果删除挂载点目录文件 file1，会发生什么情况呢？

```
root@cka-worker1:~/aufs-test/mountedfs# rm file1
root@cka-worker1:~/aufs-test/mountedfs# ls -al ../*
../layer1:
total 16
drwxr-xr-x 2 root root 4096 Jan 28 00:41 .
drwxr-xr-x 6 root root 4096 Jan 28 00:40 ..
-rw-r--r-- 1 root root   28 Jan 28 00:41 file1
-rw-r--r-- 1 root root   28 Jan 28 00:41 file2

../layer2:
total 16
drwxr-xr-x 2 root root 4096 Jan 28 00:41 .
drwxr-xr-x 6 root root 4096 Jan 28 00:40 ..
-rw-r--r-- 1 root root   28 Jan 28 00:41 file1
-rw-r--r-- 1 root root   28 Jan 28 00:41 file3

../mountedfs:
total 20
drwxr-xr-x 4 root root 4096 Jan 28 01:31 .
drwxr-xr-x 6 root root 4096 Jan 28 00:40 ..
-rw-r--r-- 1 root root   28 Jan 28 00:41 file2
-rw-r--r-- 1 root root   28 Jan 28 00:41 file3
-rw-r--r-- 1 root root   22 Jan 28 01:23 file4

../upperlayer:
total 20
drwxr-xr-x 4 root root 4096 Jan 28 01:31 .
drwxr-xr-x 6 root root 4096 Jan 28 00:40 ..
-rw-r--r-- 1 root root   22 Jan 28 01:23 file4
-r--r--r-- 2 root root    0 Jan 28 00:47 .wh.file1
-r--r--r-- 2 root root    0 Jan 28 00:47 .wh..wh.aufs
drwx------ 2 root root 4096 Jan 28 00:47 .wh..wh.orph
drwx------ 2 root root 4096 Jan 28 00:47 .wh..wh.plnk
root@cka-worker1:~/aufs-test/mountedfs#
```

可以看到，挂载点 mountedfs 中的 file1 尽管被删除了，可是在各分支层（即文件夹 layer1 和 layer2）中的 file1 仍然得以保留，同时在分支层的上层文件夹 upperlayer 中创建了相对应的控制文件将 file1 标记为已删除，这样，file1 文件不会显示在最终挂载的文件系统中。

AUFS 文件系统驱动的关键是名为 .wh*（whiteout）的文件。在示例中，可以看到删除 file1 操作通过创建文件.wh.file1 反映在上层文件夹中，.wh.file1 指示 AUFS 在生成的挂载点中隐藏 flle1 文件。如果我们在挂载点目录重新创建 file1 时（注意文件内容和前面删除的并不一样），那么 .wh.file1

这个控制文件会被移除掉，新的 file1 所在目录是在 upperlayer 中。

```
root@cka-worker1:~/aufs-test/mountedfs# echo "new content for file1" > file1
root@cka-worker1:~/aufs-test/mountedfs# ls -al ../upperlayer/
total 24
drwxr-xr-x 4 root root 4096 Jan 28 01:52 .
drwxr-xr-x 6 root root 4096 Jan 28 00:40 ..
-rw-r--r-- 1 root root   22 Jan 28 01:52 file1
-rw-r--r-- 1 root root   22 Jan 28 01:23 file4
-r--r--r-- 1 root root    0 Jan 28 00:47 .wh..wh.aufs
drwx------ 2 root root 4096 Jan 28 00:47 .wh..wh.orph
drwx------ 2 root root 4096 Jan 28 00:47 .wh..wh.plnk
root@cka-worker1:~/aufs-test/mountedfs# cat ../layer1/file1
content for file1 in layer1
root@cka-worker1:~/aufs-test/mountedfs# cat ../layer2/file1
content for file1 in layer2
root@cka-worker1:~/aufs-test/mountedfs#
```

4.3.2 OverlayFS 和 Overlay2

那么真实的容器是如何使用 UnionFS 呢，我们启动一个容器来看看具体的情况：

```
root@cka-worker1:~/aufs-test/mountedfs# docker run --name conaufs --rm -id alpine ash
00cdd1a1f8c4dc6e7789a89d461a1b72d49ea4b342c1d8749aa78ecf5b7482ed
root@cka-worker1:~/aufs-test/mountedfs# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED         STATUS         PORTS     NAMES
00cdd1a1f8c4   alpine    "ash"     5 seconds ago   Up 4 seconds             conaufs
root@cka-worker1:~/aufs-test/mountedfs#
```

使用 mount 命令查看系统的所有挂载点，发现类型为 AUFS 只有一个就是前面的示例：

```
root@cka-worker1:~/aufs-test/mountedfs# mount | grep aufs
none on /root/aufs-test/mountedfs type aufs (rw,relatime,si=e721d156a4b363aa)
root@cka-worker1:~/aufs-test/mountedfs#
```

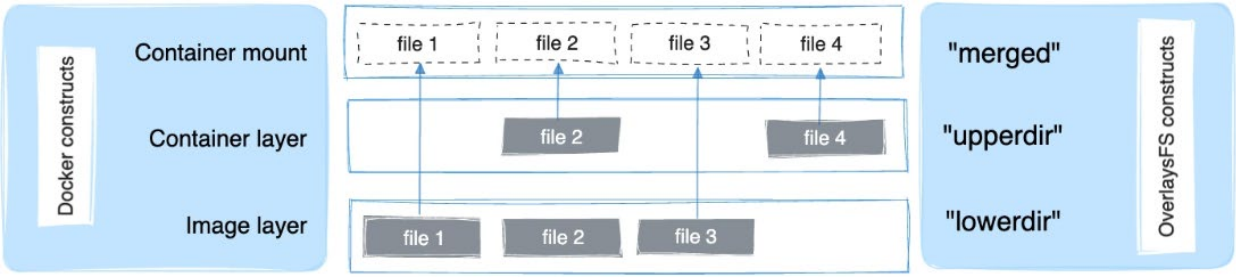
conaufs 容器其实使用的 overlay2 文件系统：

```
none on /root/aufs-test/mountedfs type aufs (rw,relatime,si=e721d156a4b363aa)
overlay on /var/lib/docker/overlay2/c7340b789a3350b83eefb662013952cbb157c4d0913d9f651e7
327ffa3b09b17/merged type overlay (rw,relatime,lowerdir=/var/lib/docker/overlay2/l/2KON
SLGIYMQWGDG4OTC3XAO6AQ:/var/lib/docker/overlay2/l/NWH6VWC3H7DAR4DCHAHQ4QIPRM,upperdir=/
var/lib/docker/overlay2/c7340b789a3350b83eefb662013952cbb157c4d0913d9f651e7327ffa3b09b1
7/diff,workdir=/var/lib/docker/overlay2/c7340b789a3350b83eefb662013952cbb157c4d0913d9f6
51e7327ffa3b09b17/work,xino=off)
nsfs on /run/docker/netns/dcbb2e116e29 type nsfs (rw)
root@cka-worker1:~/aufs-test/mountedfs#
```

Overlay2 是基于 Linux 内核的 OverlayFS 的文件系统，OverlayFS 是改进的 UnionFS，从 Linux 3.18 引入，它允许多个文件系统层 (layers) 叠加在一起，形成一个统一的视图。Overlay2 又是 OverlayFS 的改进版本，由于是直接基于内核实现，因此 Overlay2 专用于管理容器镜像和容器层的存储时，能提供比用户态文件系统（aufs）更好的性能和可扩展性。Overlay2 是 Docker 的默认存储驱动。

```
root@cka-worker1:~# docker info | grep "Storage Driver" -A 5
 Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
```

OverlayFS 的架构如下图所示：



OverlayFS 底层思想与 AUFS 相同，概念也几乎相同：将多个层组合在一起以构建联合文件系统。图中的 lowerdir 是 AUFS 中的只读层，而 upperdir 对应于读/写层。

与 AUFS 比较，OverlayFS 使用了一个 workdir 临时目录来支持 OverlayFS 文件系统的内部操作，如文件的重命名和删除等，该目录必须是空的，并且在挂载期间不能被其他进程使用

OverlayFS 仅支持将 1 个只读层与 1 个读写层合并（但可以通过嵌套叠加来实现多层叠加）。而现在 Overlay2 可支持多达 128 个层级（lower layers）。

下面通过示例来进一步理解 OverlayFS。

```
root@cka-worker1:~# mkdir overlay-test
root@cka-worker1:~# cd overlay-test/
root@cka-worker1:~/overlay-test# mkdir layer{1,2} upperlayer workdir mountedfs
root@cka-worker1:~/overlay-test# echo "content for file1 in layer1" > layer1/file1
root@cka-worker1:~/overlay-test# echo "content for file1 in layer2" > layer2/file1
root@cka-worker1:~/overlay-test# echo "content for file2 in layer1" > layer1/file2
root@cka-worker1:~/overlay-test# echo "content for file3 in layer2" > layer2/file3
root@cka-worker1:~/overlay-test# tree .
.
├── layer1
│   ├── file1
│   └── file2
├── layer2
│   ├── file1
│   └── file3
├── mountedfs
├── upperlayer
└── workdir

5 directories, 4 files
root@cka-worker1:~/overlay-test#
```

使用 overlay 挂载文件系统，挂载点 mountedfs 目录中包含的文件与我们的预期是一样的：

```
root@cka-worker1:~/overlay-test# mount -t overlay \
> -o lowerdir=layer1:layer2,upperdir=upperlayer,workdir=workdir \
> overlay mountedfs
root@cka-worker1:~/overlay-test# cd mountedfs/
root@cka-worker1:~/overlay-test/mountedfs# ls -l
total 12
-rw-r--r-- 1 root root 28 Jan 28 04:05 file1
-rw-r--r-- 1 root root 28 Jan 28 04:05 file2
-rw-r--r-- 1 root root 28 Jan 28 04:05 file3
root@cka-worker1:~/overlay-test/mountedfs# cat *
content for file1 in layer1
content for file2 in layer1
content for file3 in layer2
root@cka-worker1:~/overlay-test/mountedfs#
```

lowerdir 文件夹将按优先顺序从左到右解释，如果一个文件存在于不同的 lowerdirs 中，OverlayFS 将显示最左侧 lowerdir 中的文件。

创建一个新文件 file4 文件，再创建一个 file1 文件，这两文件都被被创建了在 upperdir 目录（不是挂载点 mountedfs），注意新建 file1 和 lowerdir:layer1,layer2 只读层的 file1 的比较。

```
root@cka-worker1:~/overlay-test/mountedfs# echo "new content for file4" > file4
root@cka-worker1:~/overlay-test/mountedfs# ls -l
total 16
-rw-r--r-- 1 root root 28 Jan 28 04:05 file1
-rw-r--r-- 1 root root 28 Jan 28 04:05 file2
-rw-r--r-- 1 root root 28 Jan 28 04:05 file3
-rw-r--r-- 1 root root 22 Jan 28 04:16 file4
root@cka-worker1:~/overlay-test/mountedfs# ls -l ../upperlayer/
total 4
-rw-r--r-- 1 root root 22 Jan 28 04:16 file4
root@cka-worker1:~/overlay-test/mountedfs# cat ../upperlayer/file4
new content for file4
root@cka-worker1:~/overlay-test/mountedfs# echo "new content for file1" > file1
root@cka-worker1:~/overlay-test/mountedfs# cat ../upperlayer/file1
new content for file1
root@cka-worker1:~/overlay-test/mountedfs# cat ../layer1/file1
content for file1 in layer1
root@cka-worker1:~/overlay-test/mountedfs# cat ../layer2/file1
content for file1 in layer2
root@cka-worker1:~/overlay-test/mountedfs#
```

4.3.3 Docer 和 Overlay2

使用 alpine 镜像创建一个容器 conoverlay 来看看其具体的存储使用情况：

```
root@cka-worker1:~# docker run --name conoverlay --rm -id alpine ash
6f8973ac28d6ca2cb7c378e244d25a4b6e033a4bf3cfd093ddf023cf1bab509e
root@cka-worker1:~# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED         STATUS         PORTS     NAMES
6f8973ac28d6   alpine    "ash"     3 seconds ago   Up 3 seconds             conoverlay
root@cka-worker1:~#
```

【简介：Alpine 是一个轻量级的 Linux 发行版，专为安全性、简单性和资源效率设计。它基于 musl libc 和 busybox，镜像体积非常小（通常只有 5MB 左右），非常适合容器化环境和资源受限的场景。Alpine 是 Docker 和 Kubernetes 等容器平台的常用基础镜像。许多企业使用 Alpine 作为微服务、CI/CD 流水线和云原生应用的基础镜像。】

容器的 id 是 6f8973ac28d6，使用 mount 查看挂载点：

```
overlay on /var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146/merge
d type overlay (rw,relatime,lowerdir=/var/lib/docker/overlay2/l/O4ZPG7K54BIGCA4O66TMYMVPDV:/var/lib/docker
/overlay2/l/NWH6VWC3H7DAR4DCHAHQ4QIPRM,upperdir=/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b
148370c91671783b5c91e0c5b017146/diff,workdir=/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148
370c91671783b5c91e0c5b017146/work,xino=off)
nsfs on /run/docker/netns/3e0de3a56a1d type nsfs (rw)
root@cka-worker1:~#
```

查看 /var/lib/docker/image/overlay2/layerdb/mounts/目录，存在一个以 6f8973ac28d6 开头的文件夹，而 6f8973ac28d6 正是容器 conoverlay 的 id。

```
root@cka-worker1:~# cd /var/lib/docker/image/overlay2/layerdb/mounts/
root@cka-worker1:/var/lib/docker/image/overlay2/layerdb/mounts# ls -l
total 8
drwxr-xr-x 2 root root 4096 Jan 28 07:17 6f8973ac28d6ca2cb7c378e244d25a4b6e033a4bf3cfd093ddf023cf1bab509e
drwxr-xr-x 2 root root 4096 Dec 16 00:43 c836387c5d4ce453d012485701c959f768a18af1eccaffa4eeeab96594ac9ae4
root@cka-worker1:/var/lib/docker/image/overlay2/layerdb/mounts#
```

进入这个目录，会发现有几个文件：

```
root@cka-worker1:/var/lib/docker/image/overlay2/layerdb/mounts# cd 6f8973ac28d6ca2cb7c378e244d25a4b6e033a4
bf3cfd093ddf023cf1bab509e
root@cka-worker1:/var/lib/docker/image/overlay2/layerdb/mounts/6f8973ac28d6ca2cb7c378e244d25a4b6e033a4bf3c
fd093ddf023cf1bab509e# ls -l
total 12
-rw-r--r-- 1 root root 69 Jan 28 07:17 init-id
-rw-r--r-- 1 root root 64 Jan 28 07:17 mount-id
-rw-r--r-- 1 root root 71 Jan 28 07:17 parent
root@cka-worker1:/var/lib/docker/image/overlay2/layerdb/mounts/6f8973ac28d6ca2cb7c378e244d25a4b6e033a4bf3c
fd093ddf023cf1bab509e# cat init-id
c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146-initroot@cka-worker1:/var/lib/docker/imag
e/overlay2/layerdb/mounts/6f8973ac28d6ca2cb7c378e244d25a4b6e033a4bf3cfd093ddf023cf1bab509e# cat mount-id
c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146root@cka-worker1:/var/lib/docker/image/ove
rlay2/layerdb/mounts/6f8973ac28d6ca2cb7c378e244d25a4b6e033a4bf3cfd093ddf023cf1bab509e# cat parent
sha256:5af4f8f59b764c64c6def53f52ada809fe38d528441d08d01c206dfb3fc3b691root@cka-worker1:/var/lib/docker/im
age/overlay2/layerdb/mounts/6f8973ac28d6ca2cb7c378e244d25a4b6e033a4bf3cfd093ddf023cf1bab509e#
```

其中 mount-id 文件包含一个 ID，该 ID 与 /var/lib/docker/overlay2/ 中的一个目录相对应，这个目录又会与 overlay2 联合文件系统相对应，最后的 overlay2 联合文件系统文件系统就是容器 6f8973ac28d6 的根文件系统。该文件夹对应于我们之前检查挂载点时暴露的挂载点。

```
root@cka-worker1:/var/lib/docker/overlay2# cd /var/lib/docker/overlay2
root@cka-worker1:/var/lib/docker/overlay2# ls -l
total 36
drwx--x--- 3 root root 4096 Dec 16 00:43 21d9cb0e3a8152dba99b808cd29509622d41a08b57a1b3403e17b0b1be0318f0
drwx--x--- 4 root root 4096 Dec 16 00:43 72f6e9d228799ee2487380000f2703c93b7c8cd31ab11da6c7e045e75d96bd91
drwx--x--- 4 root root 4096 Dec 16 00:43 72f6e9d228799ee2487380000f2703c93b7c8cd31ab11da6c7e045e75d96bd91-init
drwx--x--- 4 root root 4096 Dec 16 00:43 ba3318669b83a915acd1d15c6c702718deb49d324ce415c79337d33d71c85566
drwx--x--- 5 root root 4096 Jan 28 07:17 c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146
drwx--x--- 4 root root 4096 Jan 28 07:17 c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146-init
drwx--x--- 3 root root 4096 Jan 28 07:17 d9971bd07d03869003d3a25ae476dc90e5f4653b46a27ae12e2f39d2c6936db0
drwx--x--- 4 root root 4096 Dec 16 00:43 e94ec9340624bd28b37f269e1816027be003c8c6574537daac079322ad93c8f4
drwx------ 2 root root 4096 Jan 28 07:17 l
root@cka-worker1:/var/lib/docker/overlay2# cd c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146# ls -l
total 20
drwxr-xr-x 2 root root 4096 Jan 28 07:17 diff
-rw-r--r-- 1 root root   26 Jan 28 07:17 link
-rw-r--r-- 1 root root   57 Jan 28 07:17 lower
drwxr-xr-x 1 root root 4096 Jan 28 07:17 merged
drwx------ 3 root root 4096 Jan 28 07:17 work
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146#
```

其中的 merged 目录就是容器的挂载点目录：

```
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146# cd merged/
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146/merged# ls
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146/merged#
```

进入 conoverlay 容器，在/tmp 目录创建 tempfile：

```
root@cka-worker1:~# docker exec -it conoverlay ash
/ # pwd
/
/ # exit
root@cka-worker1:~# docker exec -it conoverlay ash
/ # ls
bin    dev    etc    home    lib    media  mnt    opt    proc    root    run    sbin    srv    sys    tmp    usr    var
/ # cd tmp/
/tmp # touch tempfile
/tmp # ls
files     tempfile
/tmp #
```

之后在主机的联合文件系统的挂载点 tmp 目录会会看到该文件：

```
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146# ls -l
total 20
drwxr-xr-x 4 root root 4096 Jan 28 07:17 diff
-rw-r--r-- 1 root root   26 Jan 28 07:17 link
-rw-r--r-- 1 root root   57 Jan 28 07:17 lower
drwxr-xr-x 1 root root 4096 Jan 28 07:17 merged
drwx------ 3 root root 4096 Jan 28 07:17 work
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146# cd merged/
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146/merged# ls
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146/merged# cd tmp/
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146/merged/tmp# ls
files  tempfile
root@cka-worker1:/var/lib/docker/overlay2/c64f5d0dbaf0ee2592e763e01d4d05c5b148370c91671783b5c91e0c5b017146/merged/tmp#
```

4.3.4 Docer 镜像的分层

Image 是通过一个定义好的 Dockerfile，使用 docker build 命令来构建的。

Dockerfile 中的每一条命令的执行结果都会成为镜像中的一个 Layer。

下面通过一个镜像的构建过程，来观察 Image 的分层机制。

示例程序使用 Python 镜像实现一个简单的线性回归机器学习算法的实现，具体代码如下：

```
root@cka-worker1:~# cat Dockerfile
# 使用官方 Python 3.12 镜像作为基础镜像
FROM python:3.12-slim

# 设置工作目录
WORKDIR /app

# 设置环境变量，使用阿里云镜像源
ENV PIP_INDEX_URL=https://mirrors.aliyun.com/pypi/simple/
ENV PIP_TRUSTED_HOST=mirrors.aliyun.com

# 复制 requirements.txt 文件到容器中
COPY requirements.txt .

# 安装依赖
RUN pip install --no-cache-dir -r requirements.txt

# 复制当前目录下的所有文件到容器的工作目录
COPY . .

# 暴露端口（如果需要）
EXPOSE 8888

# 设置默认命令（例如启动 Jupyter Notebook）
CMD ["jupyter", "notebook", "--ip=0.0.0.0", "--port=8888", "--no-browser", "--allow-root"]
root@cka-worker1:~#
```

在 Dockerfile 中包含 5 条命令，其中包括要在 Python 镜像中安装机器学习库必须的包。

```
root@cka-worker1:~# cat requirements.txt
numpy
pandas
scikit-learn
jupyter
root@cka-worker1:~#
```

【numpy：用于高效的数值计算和多维数组操作，是科学计算的基础。pandas：用于数据处理和分析，特别适合处理结构化数据。scikit-learn：用于机器学习，提供了从数据预处理到模型训练和评估的完整工具。jupyter notebook 是一个开源的交互式开发环境，广泛用于数据分析、机器学习、科学计算和教育等领域。】

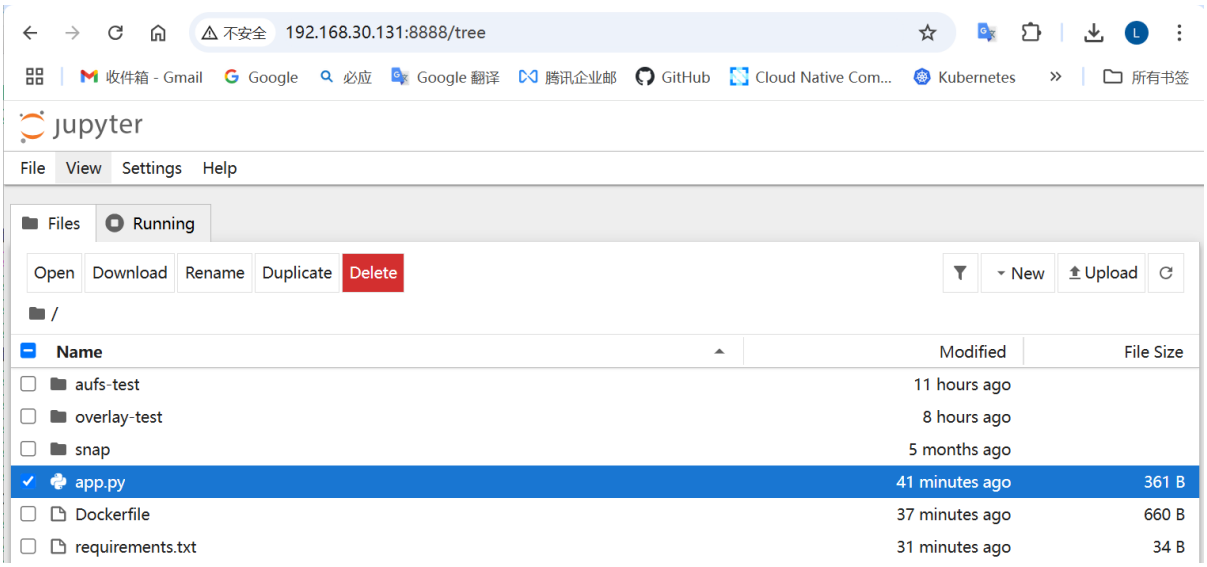
```
root@cka-worker1:~# cat app.py
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# 创建一个简单的数据集
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([1, 3, 2, 3, 5])

# 创建线性回归模型
model = LinearRegression()
model.fit(X, y)

# 进行预测
predictions = model.predict([[6]])
print("Prediction for input 6:", predictions)
root@cka-worker1:~#
```

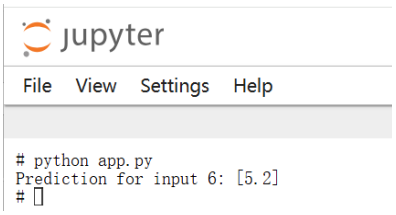
```
root@cka-worker1:~# docker build -t my-ml-app .
[+] Building 125.4s (10/10) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                              0.0s
 => => transferring dockerfile: 699B                                                                              0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                               0.6s
 => [internal] load .dockerignore                                                                                 0.0s
 => => transferring context: 2B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:69ce3aed05675d284bee807e7c45e560e98db21fb1e4c670252b4ee0f2  0.0s
 => [internal] load build context                                                                                 0.0s
 => => transferring context: 12.16kB                                                                              0.0s
 => CACHED [2/5] WORKDIR /app                                                                                     0.0s
 => [3/5] COPY requirements.txt .                                                                                 0.1s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                    119.9s
 => [5/5] COPY . .                                                                                                0.1s
 => exporting to image                                                                                            4.7s
 => => exporting layers                                                                                           4.7s
 => => writing image sha256:728923bad1e7e9007909fef923e0bbd8b6bc961e49f833d1ce5ccfaeb2bfd337                      0.0s
 => => naming to docker.io/library/my-ml-app                                                                      0.0s
root@cka-worker1:~#
```

通过构建结果可以看出，构建的过程就是执行 Dockerfile 文件中写入的命令。构建一共进行了 5 个步骤，每个步骤进行完都会生成一个随机的 ID，来标识这一 layer 中的内容。

运行新构建的容器，并通过 jupyter noteboot 得知线性回归的预测结果为 5.2。

```
root@cka-worker1:~# docker run -it --rm -p 8888:8888 my-ml-app
[I 2025-01-28 12:24:17.399 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2025-01-28 12:24:17.409 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2025-01-28 12:24:17.420 ServerApp] jupyterlab | extension was successfully linked.
[I 2025-01-28 12:24:17.428 ServerApp] notebook | extension was successfully linked.
[I 2025-01-28 12:24:17.430 ServerApp] Writing Jupyter server cookie secret to /root/.local/share/jupyter/
```





上述由 my-ml-app 镜像所创建容器的 overlay2 文件系统请自行在宿主机上进项验证。

4.4 Namespace 详解

在 Linux 系统中，namespace 是在内核级别以一种抽象的形式来封装系统资源的，通过将系统资源放在不同的 namespace 中，来实现资源隔离的目的。设置了不同 namespace 的程序，就可以享有彼此独立的一份系统资源。

Linux 中当前可用的命名空间如下：

```
root@cka-worker1:~# ls -l /proc/$$/ns
total 0
lrwxrwxrwx 1 root root 0 Jan 29 07:58 cgroup -> 'cgroup:[4026531835]'
lrwxrwxrwx 1 root root 0 Jan 29 07:58 ipc -> 'ipc:[4026531839]'
lrwxrwxrwx 1 root root 0 Jan 29 07:58 mnt -> 'mnt:[4026531840]'
lrwxrwxrwx 1 root root 0 Jan 29 07:58 net -> 'net:[4026531992]'
lrwxrwxrwx 1 root root 0 Jan 29 07:58 pid -> 'pid:[4026531836]'
lrwxrwxrwx 1 root root 0 Jan 29 07:58 pid_for_children -> 'pid:[4026531836]'
lrwxrwxrwx 1 root root 0 Jan 29 07:58 user -> 'user:[4026531837]'
lrwxrwxrwx 1 root root 0 Jan 29 07:58 uts -> 'uts:[4026531838]'
root@cka-worker1:~#
```

当前系统使用到的命名空间有，**cgroup** ns：隔离 CGroup 视图。**ipc** ns：隔离进程间通信资源。**mnt** ns：隔离文件系统挂载点。**net** ns：隔离网络资源。**pid** ns：隔离进程 ID。**user** ns：隔离用户和用户组 ID。**uts** ns：隔离主机名和域名。pid_for_children：当前进程的子进程将继承的 PID ns。

下面通过从非管理账户 user1 的角度来具体理解命名空间的工作原理。

首先配置 user1 用户的 sudo 免密，并添加 dockers 组，这组操作请自行完成。

以 user1 用户登录新的会话，运行一个名字 conns 的 alpine 容器：

```
user1@cka-worker1:~$ docker run --name conns --rm -id alpine ash
d53e97ea0ac12c9a91c51c8bbfcafe7463a1ddcc01a592dd6a303cc736937bce
user1@cka-worker1:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED          STATUS          PORTS     NAMES
d53e97ea0ac1   alpine    "ash"     6 seconds ago    Up 5 seconds              conns
user1@cka-worker1:~$
```

我们知道，conns 容器包含有一个精简但完整的文件系统：

```
user1@cka-worker1:~$ docker exec -it conns ash
/ # ls -l
total 56
drwxr-xr-x    2 root     root          4096 Dec  7  2023 bin
drwxr-xr-x    5 root     root           340 Jan 29 08:17 dev
drwxr-xr-x    1 root     root          4096 Jan 29 08:17 etc
drwxr-xr-x    2 root     root          4096 Dec  7  2023 home
drwxr-xr-x    7 root     root          4096 Dec  7  2023 lib
drwxr-xr-x    5 root     root          4096 Dec  7  2023 media
drwxr-xr-x    2 root     root          4096 Dec  7  2023 mnt
drwxr-xr-x    2 root     root          4096 Dec  7  2023 opt
dr-xr-xr-x  285 root     root             0 Jan 29 08:17 proc
drwx------    1 root     root          4096 Jan 29 08:23 root
drwxr-xr-x    2 root     root          4096 Dec  7  2023 run
drwxr-xr-x    2 root     root          4096 Dec  7  2023 sbin
drwxr-xr-x    2 root     root          4096 Dec  7  2023 srv
dr-xr-xr-x   13 root     root             0 Jan 29 08:17 sys
drwxrwxrwt    2 root     root          4096 Dec  7  2023 tmp
drwxr-xr-x    7 root     root          4096 Dec  7  2023 usr
drwxr-xr-x   12 root     root          4096 Dec  7  2023 var
/ #
```

将 conns 容器的文件系统导出：

```
user1@cka-worker1:~$ mkdir mancontainer
user1@cka-worker1:~$ cd mancontainer
user1@cka-worker1:~/mancontainer$ docker export conns -o dockercontainer.tar
user1@cka-worker1:~/mancontainer$ ls -l
total 7492
-rw------- 1 user1 docker 7669760 Jan 29 08:31 dockercontainer.tar
user1@cka-worker1:~/mancontainer$
```

在当前目录创建一个 rootfs 目录，并将导出的 conns 文件系统解压到这个目录：

```
user1@cka-worker1:~/mancontainer$ mkdir rootfs
user1@cka-worker1:~/mancontainer$ tar xf dockercontainer.tar -C rootfs/
user1@cka-worker1:~/mancontainer$ ls -l rootfs/
total 68
drwxr-xr-x  2 user1 docker 4096 Dec  7  2023 bin
drwxr-xr-x  4 user1 docker 4096 Jan 29 08:17 dev
drwxr-xr-x 19 user1 docker 4096 Jan 29 08:17 etc
drwxr-xr-x  2 user1 docker 4096 Dec  7  2023 home
drwxr-xr-x  7 user1 docker 4096 Dec  7  2023 lib
drwxr-xr-x  5 user1 docker 4096 Dec  7  2023 media
drwxr-xr-x  2 user1 docker 4096 Dec  7  2023 mnt
drwxr-xr-x  2 user1 docker 4096 Dec  7  2023 opt
dr-xr-xr-x  2 user1 docker 4096 Dec  7  2023 proc
drwx------  2 user1 docker 4096 Jan 29 08:23 root
drwxr-xr-x  2 user1 docker 4096 Dec  7  2023 run
drwxr-xr-x  2 user1 docker 4096 Dec  7  2023 sbin
drwxr-xr-x  2 user1 docker 4096 Dec  7  2023 srv
drwxr-xr-x  2 user1 docker 4096 Dec  7  2023 sys
drwxrwxr-x  2 user1 docker 4096 Dec  7  2023 tmp
drwxr-xr-x  7 user1 docker 4096 Dec  7  2023 usr
drwxr-xr-x 12 user1 docker 4096 Dec  7  2023 var
user1@cka-worker1:~/mancontainer$
```

这么做的目的就是要手工创建一个资源隔离的类似容器对象。

接下来使用 chroot 命令把当前 shell 的 root 切换到刚创建的文件系统。

```
user1@cka-worker1:~/mancontainer$ sudo chroot $PWD/rootfs ash
/ # which ls
/bin/ls
/ # ls -l /bin/ls
lrwxrwxrwx    1 1000     998             12 Dec  7  2023 /bin/ls → /bin/busybox
/ # id
uid=0(root) gid=0(root) groups=0(root)
/ # whoami
root
/ #
```

好了，新的文件系统好像是独立的 root 文件系统，但是看 ip link 命令显示的网络信息：

```
/ # ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP qlen 1000
    link/ether 00:0c:29:00:8c:55 brd ff:ff:ff:ff:ff:ff
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:32:8b:de:3c brd ff:ff:ff:ff:ff:ff
5: veth1baf6c2@if4: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue master docker0 state UP
    link/ether 62:be:ae:53:c7:5b brd ff:ff:ff:ff:ff:ff
/ # ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP qlen 1000
    link/ether 00:0c:29:00:8c:55 brd ff:ff:ff:ff:ff:ff
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:32:8b:de:3c brd ff:ff:ff:ff:ff:ff
5: veth1baf6c2@if4: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue master docker0 state UP
    link/ether 62:be:ae:53:c7:5b brd ff:ff:ff:ff:ff:ff
```

再看系统进程信息：

```
/ # mount -t proc proc /proc && ps -ef
PID   USER     TIME  COMMAND
    1 root      0:05 {systemd} /sbin/init maybe-ubiquity
    2 root      0:00 [kthreadd]
    3 root      0:00 [rcu_gp]
    4 root      0:00 [rcu_par_gp]
    6 root      0:00 [kworker/0:0H-kb]

 1836 1000      0:00 sshd: user1@notty
 1837 1000      0:00 sshd: user1@pts/1
```

可以发现，除了挂载点之外的网络、进程等资源在“模拟容器”中都能看到，这些资源没有隔离。

回到 user1 用户，使用 unshare 命令隔离出一组新的命名空间集合。unshare 命令用于创建一个新 ns 集合，并在其中启动一个 Bash shell，实现资源隔离，这是 Linux 容器技术的基础之一。

```
user1@cka-worker1:~/mancontainer$ unshare --mount --uts --ipc --net \
> --pid --fork --user --map-root-user /bin/bash
root@cka-worker1:~/mancontainer# echo $$
1
root@cka-worker1:~/mancontainer# id
uid=0(root) gid=0(root) groups=0(root),65534(nogroup)
root@cka-worker1:~/mancontainer# ip link
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
root@cka-worker1:~/mancontainer#
```

```
root@cka-worker1:~/mancontainer# ps -ef | grep bash
nobody      1596    1417  0 08:11 pts/0    00:00:00 -bash
root        1839    1837  0 08:11 pts/1    00:00:00 -bash
nobody      2007    2006  0 08:16 pts/0    00:00:00 bash
root        2048    2047  0 08:17 pts/1    00:00:00 bash
root        3585    2048  0 09:08 pts/1    00:00:00 unshare --mount --uts --ipc --net --pid
--fork --user --map-root-user /bin/bash
root        3591    3585  0 09:08 pts/1    00:00:00 /bin/bash
root        3721    3591  0 09:13 pts/1    00:00:00 grep --color=auto bash
root@cka-worker1:~/mancontainer#
```

```
root@cka-worker1:~/mancontainer# ps -ef | grep 2048
root        2048    2047  0 08:17 pts/1    00:00:00 bash
root        3585    2048  0 09:08 pts/1    00:00:00 unshare --mount --uts --ipc --net --pid
--fork --user --map-root-user /bin/bash
root        3832    3591  0 09:17 pts/1    00:00:00 grep --color=auto 2048
root@cka-worker1:~/mancontainer#
```

特别要说明的是，如果此时我们在其他终端上检查主机中的进程时，会看到，即使上面的用户 root，在进程之外，也是在普通用户 user1 的凭据下执行的：

```
root@cka-worker1:~# ps -ef | grep 2048
user1       2048    2047  0 08:17 pts/1    00:00:00 bash
user1       3585    2048  0 09:08 pts/1    00:00:00 unshare --mount --uts --ipc --net --pid
--fork --user --map-root-user /bin/bash
root        3975    2007  0 09:24 pts/0    00:00:00 grep --color=auto 2048
root@cka-worker1:~#
```

这就是 PID 命名空间的神奇之处，它使得一个进程具有不同的 PID 号，例如 K8s pod 场景。

回到“模拟容器”环境，如果想要显示当前正在运行的进程，结果先显示的是主机中的所有进程：

```
root@cka-worker1:~/mancontainer# ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
nobody         1       0  0 08:10 ?        00:00:05 /sbin/init maybe-ubiquity
nobody         2       0  0 08:10 ?        00:00:00 [kthreadd]
nobody         3       2  0 08:10 ?        00:00:00 [rcu_gp]
nobody         4       2  0 08:10 ?        00:00:00 [rcu_par_gp]
nobody         6       2  0 08:10 ?        00:00:00 [kworker/0:0H-kblockd]
nobody         8       2  0 08:10 ?        00:00:00 [mm_percpu_wq]
nobody         9       2  0 08:10 ?        00:00:00 [ksoftirqd/0]
nobody        10       2  0 08:10 ?        00:00:14 [rcu_sched]
```

我们知道，进程本质上是/proc 挂载点中的文件描述符，因此需要将 procfs 虚拟文件系统挂载到当前的/proc 目录，就可在新的 PID 命名空间中实现进程的隔离：

```
root@cka-worker1:~/mancontainer# mount -t proc proc /proc
root@cka-worker1:~/mancontainer# ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 09:08 pts/1    00:00:00 /bin/bash
root          34       1  0 09:35 pts/1    00:00:00 ps -ef
root@cka-worker1:~/mancontainer#
```

继续，我们在容器外部创建一个网络设备并将其放入命名空间：

```
user1@cka-worker1:~$ sudo ip link add vethLocal type veth peer name vethContainer
user1@cka-worker1:~$ sudo ip link set vethLocal up
user1@cka-worker1:~$ sudo ip link set vethContainer up
user1@cka-worker1:~$ ps -ef | grep '/bin/bash'
user1       3585    2048  0 09:08 pts/1    00:00:00 unshare --mount --uts --ipc --net --pid
--fork --user --map-root-user /bin/bash
user1       3591    3585  0 09:08 pts/1    00:00:00 /bin/bash
user1       4884    4774  0 09:52 pts/0    00:00:00 grep --color=auto /bin/bash
user1@cka-worker1:~$ sudo ip link set vethContainer netns 3591
user1@cka-worker1:~$
```

回到“模拟容器”环境，会看到有一个新的网络设备出现了：

```
root@cka-worker1:~/mancontainer# ip link
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
6: vethContainer@if7: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT
group default qlen 1000
    link/ether 5e:9a:f8:e3:e6:52 brd ff:ff:ff:ff:ff:ff link-netnsid 0
root@cka-worker1:~/mancontainer#
```

4.5 CGroups 资源控制组

通过命名空间可以实现进程间在文件系统、网络以及宿主进程之间的相互隔离，但是对于宿主机的物理资源比如 CPU、内存、块设备等，如果同一机器上运行了多个相互独立的容器，这些容器在执行高负载任务时必定会因此产生严重的资源争用问题。比如，当其中的某个容器正在执行 CPU 密集型的任务，此时如果没有某种限制，这个容器必定会影响在同一主机上运行的其他容器中任务，进而导致这些容器的性能受到极大的影响，这在生产上是必须要避免的。

解决进程间虚拟资源隔离问题的办法就是 CGroups（Control Groups），它是 Linux 内核的一种机制，用于对系统资源（如 CPU、内存、磁盘 I/O、网络等）进行分组管理和限制。CGroups 允许系统管理员将**进程分组**，并为每个组分配特定的资源限制，从而实现资源的隔离、控制和监控。

CGroups 的核心功能包括：

1. 限制组内进程的资源使用量（如 CPU 使用率、内存使用量等）。防止某个进程或组占用过多资源，影响其他进程或系统的稳定性。
2. 为不同的组分配不同的资源优先级。为高优先级任务分配更多的 CPU 时间。
3. 监控和记录组内进程的资源使用情况（如 CPU 时间、内存使用量等）。
4. 对组内进程进行挂起、恢复、终止等操作。
5. CGroups 支持层次化结构，可以将组嵌套在其他组中，形成树状结构。

CGroups 通过子系统（subsystem）来管理不同类型的资源。常见的子系统包括：

```
root@cka-worker1:~# ls /sys/fs/cgroup/
blkio    cpu,cpuacct  freezer  net_cls          perf_event  systemd
cpu      cpuset       hugetlb  net_cls,net_prio pids        unified
cpuacct  devices      memory   net_prio         rdma
root@cka-worker1:~#
```

其中，**cpu**：限制 CPU 使用率，支持按比例分配 CPU 时间。**memory**：限制内存使用量，支持设置内存和交换空间的上限。**blkio**：限制块设备如磁盘的 I/O 操作。**devices**：控制组内进程对设备

的访问权限。**freezer**：挂起或恢复组内的进程。**net_cls**：标记网络数据包，用于流量控制。**pids**：限制组内进程的数量。**cpuacct**：统计组内进程的 CPU 使用情况。

下面通过示例来演示 CGroups 对进程的内存资源使用的限制方法。

演示程序不间断地从 /dev/urandom 设备读取随机数据并追加到主内存缓冲区中，同时输出当前累积的内存消耗，程序会一直运行，直到内存耗尽或手动终止。完整 C 实现如下，gcc 编译环境请自行安装。使用 gcc -o cgmemdemo cgmemdemo.c 命令编译。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // Open the random device in binary mode
    FILE *f = fopen("/dev/urandom", "rb");
    if (!f) {
        perror("Failed to open /dev/urandom");
        return 1;
    }

    char *data_ptr = NULL;          // Dynamic memory pointer
    size_t total_size = 0;          // Total memory size
    int i = 0;
    const size_t chunk_size = 10 * 1024 * 1024;  // 10MB

    while (1) {
        // 1. Allocate a temporary buffer
        char *temp = malloc(chunk_size);
        if (!temp) {
            perror("Failed to allocate temporary buffer");
            break;
        }

        // 2. Read 10MB of data from the device
        size_t bytes_read = fread(temp, 1, chunk_size, f);
        if (bytes_read != chunk_size) {
            perror("Failed to read data");
            free(temp);
            break;
        }

        // 3. Expand the main memory buffer
        char *new_ptr = realloc(data_ptr, total_size + chunk_size);
        if (!new_ptr) {
            perror("Failed to expand memory");
            free(temp);
            break;
        }
        data_ptr = new_ptr;
        memcpy(data_ptr + total_size, temp, chunk_size);  // Append data
        total_size += chunk_size;

        free(temp);  // Free the temporary buffer

        // 4. Print the current memory consumption
        i++;
        printf("%dmb\n", i * 10);
        fflush(stdout);  // Force flush the output buffer
    }

    // Clean up resources
    free(data_ptr);
    fclose(f);
    return 0;
}
```

我们将在两个 shell 会话中同时运行这个程序，不出意外的话，主机内存很快就会耗尽。为防止这

种情况的出现，可以创建一个 CGroups 来限制这两个 shell 进程的内存使用，当内存达到限制上限时终止程序，从而保证主机内存资源的安全使用。

创建 CGroups 很简单，只需在控制组文件系统创建一个目录即可。在本例中，我们将创建一个名为“demo”的内存组，之后内核会自动将可用的子系统配置文件填充到这个目录：

```
root@cka-worker1:~# mkdir /sys/fs/cgroup/memory/demo
root@cka-worker1:~# ls /sys/fs/cgroup/memory/demo/
cgroup.clone_children           memory.kmem.tcp.failcnt             memory.oom_control
cgroup.event_control            memory.kmem.tcp.limit_in_bytes      memory.pressure_level
cgroup.procs                    memory.kmem.tcp.max_usage_in_bytes  memory.soft_limit_in_bytes
memory.failcnt                  memory.kmem.tcp.usage_in_bytes      memory.stat
memory.force_empty              memory.kmem.usage_in_bytes          memory.swappiness
memory.kmem.failcnt             memory.limit_in_bytes               memory.usage_in_bytes
memory.kmem.limit_in_bytes      memory.max_usage_in_bytes           memory.use_hierarchy
memory.kmem.max_usage_in_bytes  memory.move_charge_at_immigrate     notify_on_release
memory.kmem.slabinfo            memory.numa_stat                    tasks
root@cka-worker1:~#
```

将 demo 内存控制组的最大内存使用量限制在 100M，为演示效果考虑，禁止使用交换分区：

```
root@cka-worker1:~# echo "100000000" > /sys/fs/cgroup/memory/demo/memory.limit_in_bytes
root@cka-worker1:~# echo "0" > /sys/fs/cgroup/memory/demo/memory.swappiness
root@cka-worker1:~#
```

启动的两个 bash 进程 id 显示如下：

```
root@cka-worker1:~# ps -ef | grep bash
root         1803    1570  0 02:47 pts/0    00:00:00 -bash
root         1939    1827  0 02:47 pts/1    00:00:00 -bash
root        20468    1803  0 03:25 pts/0    00:00:00 grep --color=auto bash
root@cka-worker1:~#
```

将这两个进程加入到 demo 内存控制组的 tasks 文件中：

```
root@cka-worker1:~# ps
    PID TTY          TIME CMD
   1803 pts/0    00:00:00 bash
  20584 pts/0    00:00:00 ps
root@cka-worker1:~# echo $$ >> /sys/fs/cgroup/memory/demo/tasks
root@cka-worker1:~#
```

```
root@cka-worker1:~# ps
    PID TTY          TIME CMD
   1939 pts/1    00:00:00 bash
  20649 pts/1    00:00:00 ps
root@cka-worker1:~# echo $$ >> /sys/fs/cgroup/memory/demo/tasks
root@cka-worker1:~#
```

可以看到，当两个应用的内存使用达到 100M 时，demo 内存控制组会终止程序的运行：

```
root@cka-worker1:~# ps
    PID TTY          TIME CMD
   1803 pts/0    00:00:00 bash
  20746 pts/0    00:00:00 ps
root@cka-worker1:~# ./cgmemdemo
10mb
20mb
30mb
40mb
50mb
60mb
70mb
80mb
Killed
root@cka-worker1:~#
```

```
root@cka-worker1:~# ps
    PID TTY          TIME CMD
   1939 pts/1    00:00:00 bash
  20757 pts/1    00:00:00 ps
root@cka-worker1:~# ./cgmemdemo
10mb
20mb
30mb
40mb
50mb
60mb
70mb
80mb
Killed
root@cka-worker1:~#
```

如果要取消控制组的限制，只要删除控制目录即可：

```
root@cka-worker1:~# rmdir /sys/fs/cgroup/memory/demo/
root@cka-worker1:~#
```

注意：只有 tasks 文件中的所有进程都已退出或已重新分配给另一个组才能删除。

CGroup 的前身是出自于 2006 年 Google 工程师 Paul Menage 和 Rohit Seth 启动的一个名字叫做 process containers 的项目，后来因为与 container 在内核中的名字有冲突，因此在 2007 年改名为 control groups，合并到 2008 年发布的 2.6.24 内核版本中。

最初 CGroups 的版本被称为 v1，后续的开发工作由 Tejun Heo 接管，他重新设计并重写了 CGroups，新版本被称为 v2，并在 kernel 4.5 版本中引入。随着 v2 版本的普及，容器资源的管理变得更加高效和灵活。

在容器环境中，CGroup 扮演着至关重要的角色，它是容器资源管理和隔离的核心技术之一。在 Docker 和 Kubernetes 中，CGroup 被广泛用于限制容器的 CPU、内存、磁盘 I/O 等资源。

4.6 Docker 网络

Docker 容器在创建时，虽然会通过 Linux 的命名空间完成与宿主机进程的网络隔离，但是却有没有办法通过宿主机的网络与整个互联网相连，这会对 Docker 的应用产生限制。为此，每一个使用 docker run 启动的容器都会为其分配单独的网络命名空间。Docker 提供了四种不同的网络模式，分别是 Host、Container、None 和 Bridge 模式。

4.6.1 Docker 网络模式

1. Host 模式

在 Host 模式下，容器直接使用宿主机的网络栈，不进行网络隔离。这意味着容器的网络配置与宿主机完全相同，容器内的应用程序可以直接使用宿主机的 IP 地址和端口。

```
root@cka-worker1:~# docker run --network host nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2025/03/20 06:30:40 [notice] 1#1: using the "epoll" event method
2025/03/20 06:30:40 [notice] 1#1: nginx/1.27.4
```

```
root@cka-worker1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS     NAMES
c8bf123694e8   nginx     "/docker-entrypoint.…"   2 minutes ago    Up 2 minutes              nice_herschel
a8772d78b8b9   alpine    "ash"                    17 minutes ago   Up 17 minutes             conaufs
root@cka-worker1:~# curl http://127.0.0.1:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

```
2025/03/20 06:30:40 [notice] 1#1: start worker process 30
127.0.0.1 - - [20/Mar/2025:06:33:17 +0000] "GET / HTTP/1.1" 200 615 "-" "curl/7.68.0" "-"
```

如果在宿主机上访问 http://127.0.0.1:80，实际上访问的是容器内的 Nginx 服务。

Host 模式适用场景：需要容器与宿主机共享网络栈，适用于对网络性能要求较高的场景，此时不需要端口映射，容器直接可以使用宿主机的端口。

2. Container 模式

在 Container 模式下，新创建的容器会共享另一个容器的网络命名空间。这意味着两个容器的网络配置完全相同，它们可以通过 localhost 直接通信。

```
root@cka-worker1:~# docker run -d --name nginx nginx
aef05c6e3264c97442d7416c70fc3cc833d4506d3f70454d785216efc67cdc69
root@cka-worker1:~# docker run --network container:nginx alpine ping localhost
PING localhost (::1): 56 data bytes
64 bytes from ::1: seq=0 ttl=64 time=0.021 ms
64 bytes from ::1: seq=1 ttl=64 time=0.049 ms
64 bytes from ::1: seq=2 ttl=64 time=0.156 ms
64 bytes from ::1: seq=3 ttl=64 time=0.090 ms
```

```
root@cka-worker1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS     NAMES
9ba17da4ea69   alpine    "ping localhost"         22 seconds ago   Up 22 seconds             stupefied_rosalind
aef05c6e3264   nginx     "/docker-entrypoint.…"   44 seconds ago   Up 44 seconds   80/tcp    nginx
root@cka-worker1:~#
```

通过 --network container:nginx 参数，Alpine 容器就可共享 nginx 容器的网络命名空间，通过 localhost 就可直接访问 nginx 容器的网络服务。

Container 模式适用场景：需要多个容器共享同一个网络栈，适用于需要紧密协作的容器组。例如，一个容器运行应用程序，另一个容器运行日志收集器，两者就可以通过 localhost 直接通信。

3. None 模式

在 None 模式下，容器不会配置任何网络栈，即容器内没有网络接口，无法与外界通信。

```
root@cka-worker1:~# docker run --network none alpine ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
root@cka-worker1:~#
```

使用 --network none 参数启动一个 Alpine 容器，在容器内执行 ip addr 命令，可以看到容器内没有任何网络接口。

```
root@cka-worker1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS     NAMES
aef05c6e3264   nginx     "/docker-entrypoint.…"   10 minutes ago   Up 10 minutes   80/tcp    nginx
root@cka-worker1:~#
```

容器无法与外界通信，也无法被外界访问。

None 模式适用场景：适用于不需要网络通信的场景，例如只进行本地数据处理或测试；安全性要求较高的场景，确保容器与外界完全隔离。

4. Bridge 模式

Bridge 模式是 Docker 的默认网络模式。在这种模式下，Docker 会为每个容器创建一个虚拟网络接口，并将其连接到 Docker 的虚拟网桥（通常是 docker0）。容器通过这个网桥与宿主机和其他容

器进行通信。

```
root@cka-worker1:~# docker run -d --name nginx -p 8080:80 nginx
d89da2e42ddd97b0fb5308d2ab5a9fa0063981e39a117684ef6f0abd1403a187
root@cka-worker1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                       NAMES
d89da2e42ddd   nginx     "/docker-entrypoint.…"   3 seconds ago   Up 2 seconds   0.0.0.0:8080→80/tcp, [::]:8080→80/tcp       nginx
root@cka-worker1:~#
```

使用 -p 8080:80 参数启动一个 Nginx 容器，将容器的 80 端口映射到宿主机的 8080 端口。Docker 会自动创建一个虚拟网络接口，并将其连接到 docker0 网桥。通过 curl http://localhost:8080 就能访问容器内的 Nginx 服务。

Bridge 模式适用于大多数需要容器与宿主机或其他容器通信的场景。通过端口映射，容器内的服务可以被外部网络访问。

4.6.2 Bridge 模式详解

在 Bridge 模式下，Docker 除了分配隔离的网络命名空间之外，还会为所有的容器设置 IP 地址。当 Docker 服务器在主机上启动之后会创建新的虚拟网桥 docker0，随后在该主机上启动的全部服务在默认情况下都会与该网桥相连。

docker0 网桥使用 brctl show 命令可以查看到，此时系统没有容器运行：

```
root@cka-worker1:~# docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
root@cka-worker1:~# brctl show
bridge name     bridge id               STP enabled     interfaces
docker0         8000.9a9a0bf90bf0       no
root@cka-worker1:~#
```

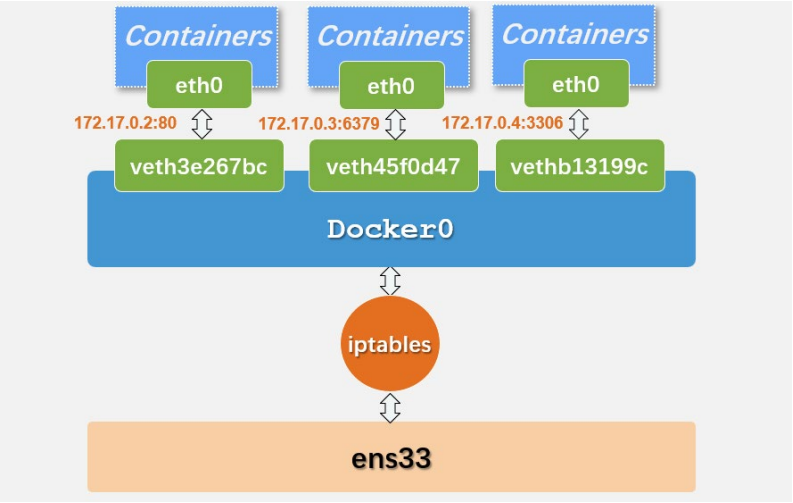
现在运行三个容器，分别是 nginx，redis，mysql，：

```
root@cka-worker1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                                                   NAMES
1703fe08b549   mysql     "docker-entrypoint.s…"   3 minutes ago   Up 3 minutes   0.0.0.0:3306→3306/tcp, [::]:3306→3306/tcp, 33060/tcp    mysql
5b4d3c1b6b39   redis     "docker-entrypoint.s…"   3 minutes ago   Up 3 minutes   0.0.0.0:6379→6379/tcp, [::]:6379→6379/tcp               redis
6fc627222788   nginx     "/docker-entrypoint.…"   4 minutes ago   Up 4 minutes   0.0.0.0:8080→80/tcp, [::]:8080→80/tcp                   nginx
root@cka-worker1:~#
```

检查网桥和 iptables，注意标注的信息：

```
root@cka-worker1:~# brctl show
bridge name     bridge id               STP enabled     interfaces
docker0         8000.aa23df03d06c       no              veth3e267bc
                                                        veth45f0d47
                                                        vethb13199c

root@cka-worker1:~# iptables -t nat -L
Chain PREROUTING (policy ACCEPT)
target     prot opt source               destination
DOCKER     all  --  anywhere             anywhere             ADDRTYPE match dst-type LOCAL

Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
DOCKER     all  --  anywhere            !localhost/8          ADDRTYPE match dst-type LOCAL

Chain POSTROUTING (policy ACCEPT)
target     prot opt source               destination
MASQUERADE  all  --  172.17.0.0/16        anywhere

Chain DOCKER (2 references)
target     prot opt source               destination
RETURN     all  --  anywhere             anywhere
DNAT       tcp  --  anywhere             anywhere             tcp dpt:http-alt to:172.17.0.2:80
DNAT       tcp  --  anywhere             anywhere             tcp dpt:6379 to:172.17.0.3:6379
DNAT       tcp  --  anywhere             anywhere             tcp dpt:mysql to:172.17.0.4:3306
root@cka-worker1:~#
```

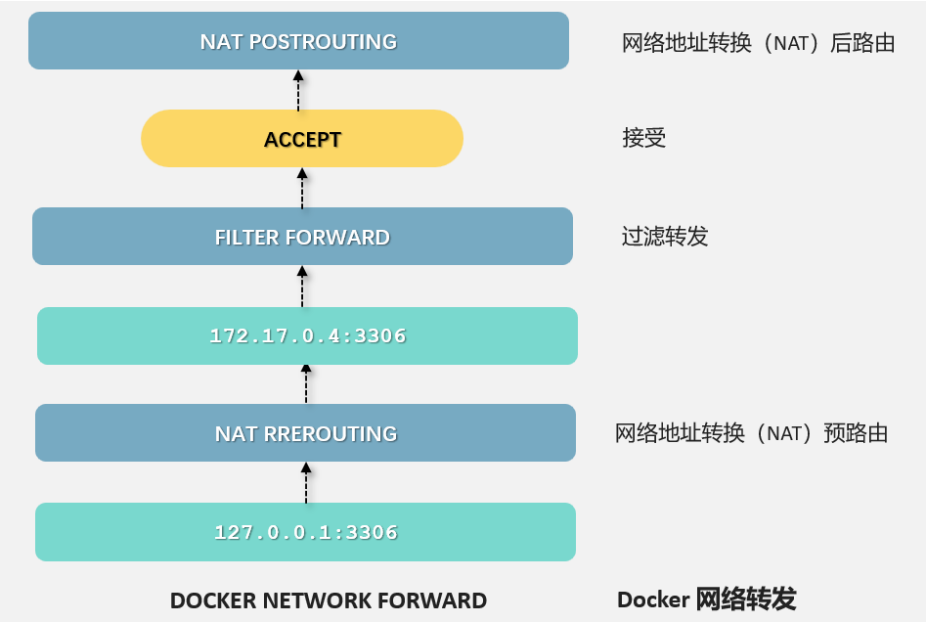
可以看出，docker0 为每一个容器分配一个新的 IP 地址并将 docker0 的 IP 地址设置为默认的网关，其拓扑关系如下图所示：



比如，mysql 容器的 ip 是 172.17.0.4，Docker 网桥规则会将从任意源发送到当前机器 3306 端口的 TCP 包都转发到 172.17.0.4:3306 所在的地址上，如果在当前机器上直接 ping 这个 IP 地址就会发现它是可以访问到的：

```
root@cka-worker1:~# ping 172.17.0.4 -c 3
PING 172.17.0.4 (172.17.0.4) 56(84) bytes of data.
64 bytes from 172.17.0.4: icmp_seq=1 ttl=64 time=0.077 ms
64 bytes from 172.17.0.4: icmp_seq=2 ttl=64 time=0.096 ms
64 bytes from 172.17.0.4: icmp_seq=3 ttl=64 time=0.057 ms

--- 172.17.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2054ms
rtt min/avg/max/mdev = 0.057/0.076/0.096/0.015 ms
root@cka-worker1:~#
```

从上述操作我们可以知道 Docker 是如何将容器的内部的端口暴露出来并对数据包进行转发的，当有 Docker 容器需要将服务暴露给宿主机器时，docker0 就为容器分配一个 ip 地址，同时向 iptables 追加一条新的转发规则，Docker 网络转发规则如下图所示：



我们可以通过测试 mysql 服务器是否正常运行来理解上述 Docker 的网络转发过程：

```
root@cka-worker1:~# mysqladmin -h 127.0.0.1 -P 3306 -u root -p ping
Enter password:
mysqld is alive
root@cka-worker1:~#
```

使用 mysqladmin 客户机在宿主机器的命令行中访问 127.0.0.1:3306 地址时，会经过 iptables 的预路由（NAT PREROUTING）将该 ip 定向到了 mysql 容器的 ip 地址 172.17.0.4，重定向过的数据包此时就可以通过 iptables 中的过滤转发（FILTER FORWARD）配置，最终在后路由（NAT POSTROUTING）阶段将容器 ip 地址伪装成主机的 127.0.0.1，到这里，虽然从外面看起来请求的是 127.0.0.1:3306，但实际上请求的已经是 Docker 容器暴露出的端口了。

在/sys/class/net 目录可以查看到以上的所有网络设备：

```
root@cka-worker1:~# ls -l /sys/class/net
total 0
lrwxrwxrwx 1 root root 0 Mar 20 11:31 docker0 → ../../devices/virtual/net/docker0
lrwxrwxrwx 1 root root 0 Mar 20 11:31 ens33 → ../../devices/pci0000:00/0000:00:11.0/0000:02:01.0/net/ens33
lrwxrwxrwx 1 root root 0 Mar 20 11:31 lo → ../../devices/virtual/net/lo
lrwxrwxrwx 1 root root 0 Mar 20 11:34 veth3e267bc → ../../devices/virtual/net/veth3e267bc
lrwxrwxrwx 1 root root 0 Mar 20 11:34 veth45f0d47 → ../../devices/virtual/net/veth45f0d47
lrwxrwxrwx 1 root root 0 Mar 20 11:34 vethb13199c → ../../devices/virtual/net/vethb13199c
root@cka-worker1:~#
```

总之，Docker 通过 Linux 的命名空间实现了网络的隔离，进而又通过 iptables 进行了数据包的转发，从而实现了 Docker 容器独立和完整的网络功能。

第 4 节 结尾。